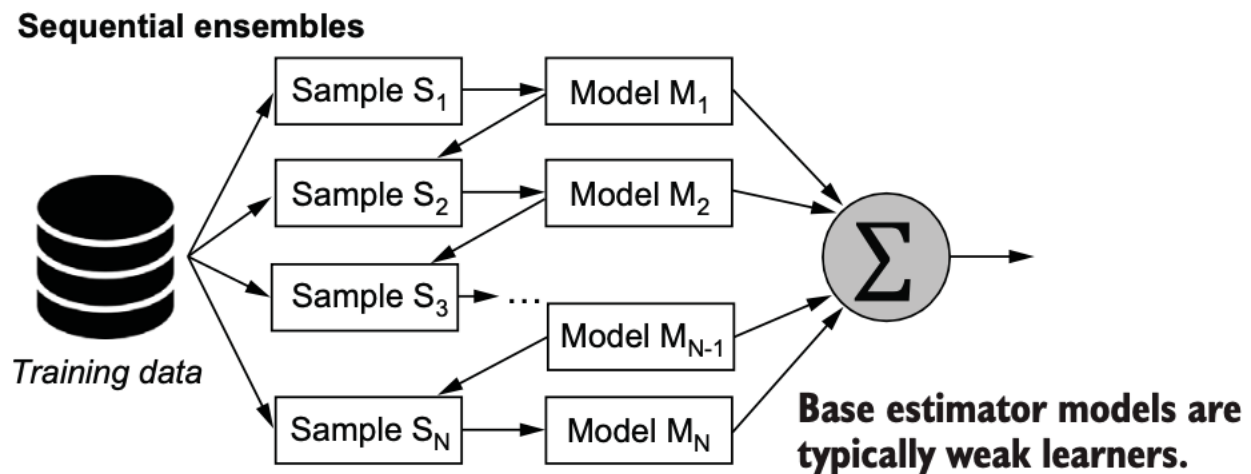
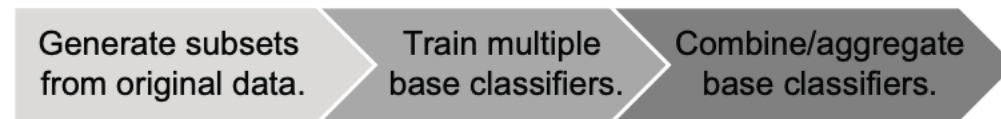
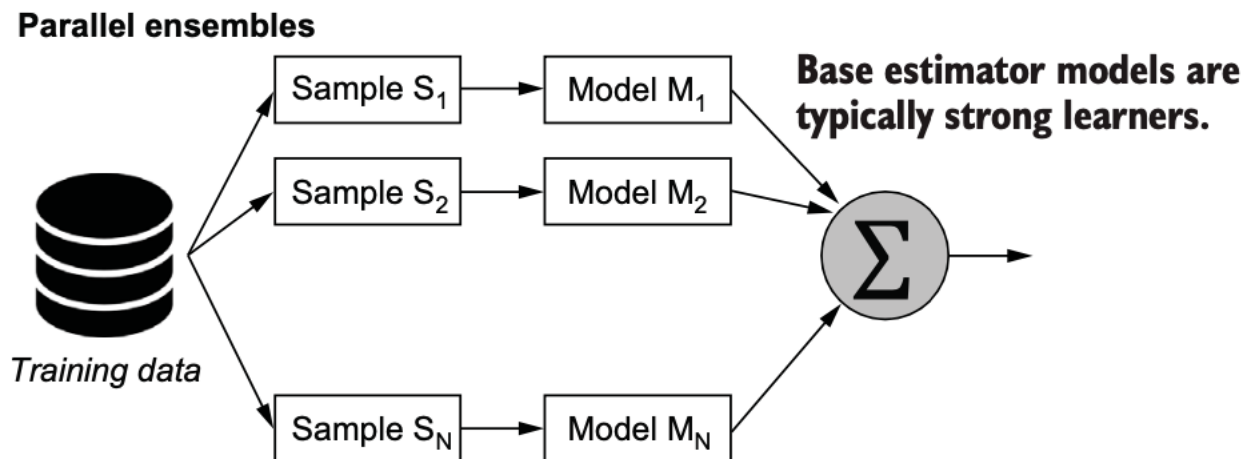


Boosting

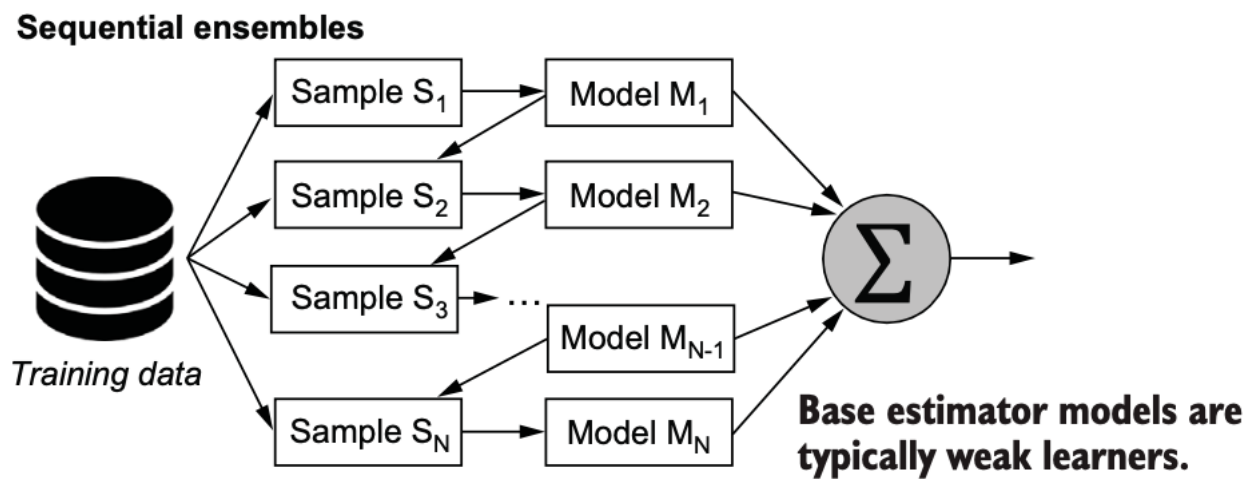
2026.04.14

Parallel vs sequential ensembles



Gradient boosting

Instead of predicting the target the learner predicts on the corrections needed



$$F_m(x) = F_0(x) + \sum_{i=1}^m \eta \cdot h_i(x)$$

Loss function

Defines how we determine error

$$\text{MSE} = (y - \hat{y})^2$$

$$\text{LogLoss} = -[y \cdot \log(p) + (1 - y) \cdot \log(1 - p)]$$

Calculating the gradients

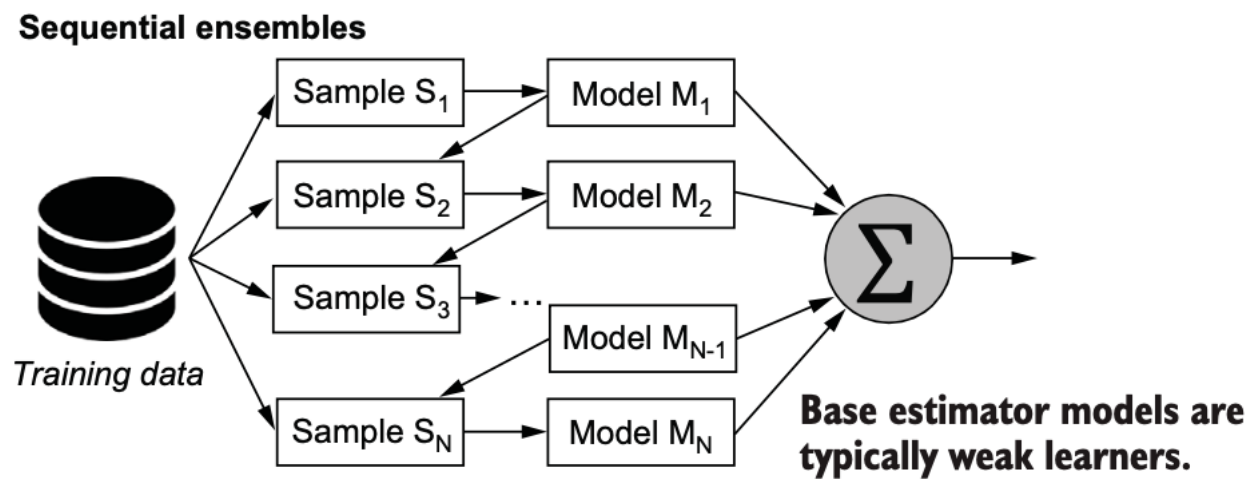
$$g_i = \frac{\partial L}{\partial \hat{y}_i}$$

$$g \approx y - \hat{y}$$

$$g \approx y - p$$

Gradient boosting

Instead of predicting the target the learner predicts the gradient



Gradient boosting rounds

Learner 0 :

$F_0(x)$ = initial prediction (averages)
calculate gradients for next round

Learner 1:

train on gradients
 $F_1(x) = F_0(x) + \eta \cdot h_1(x)$
calculate gradients for next round

Learner 2:

train on gradients
 $F_2(x) = F_1(x) + \eta \cdot h_2(x)$
calculate gradients for next round

·
·
·

Learner n:

train on gradients
 $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$

Gradient boosting

Learner 0 :

$F_0(x)$ = initial prediction (averages)
calculate gradients for next round

Learner 1:

train on gradients
 $F_1(x) = F_0(x) + \eta \cdot h_1(x)$
calculate gradients for next round

Learner 2:

train on gradients
 $F_2(x) = F_1(x) + \eta \cdot h_2(x)$
calculate gradients for next round

·
·
·

Learner n:

train on gradients
 $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$

$$F_m(x) = F_0(x) + \sum_{i=1}^m \eta \cdot h_i(x)$$

Limitations of gradient boosting

1. Slow training
2. Expensive split finding
3. Risk of overfitting
4. Sensitivity to hyperparameters

XGBoost addresses these

1. Slow training
2. Expensive split finding
3. Risk of overfitting
4. Sensitivity to hyperparameters

XGBoost has additional features

1. Correcting for class imbalance (`scale_pos_weight`)
2. Workflow for missing values
3. Encoding categorical features
4. Choice of loss functions

Model comparisons

Feature	Random Forest	XGBoost	LightGBM	CatBoost
Core approach	Bagging	Boosting	Boosting	Boosting

Model comparisons

Feature	Random Forest	XGBoost	LightGBM	CatBoost
Core approach	Bagging	Boosting	Boosting	Boosting
Tree growth	Level-wise	Level-wise	Leaf-wise (best-first)	Symmetric (oblivious)

Model comparisons

Feature	Random Forest	XGBoost	LightGBM	CatBoost
Core approach	Bagging	Boosting	Boosting	Boosting
Tree growth	Level-wise	Level-wise	Leaf-wise (best-first)	Symmetric (oblivious)
Split behavior	Balanced (best split per node + random features)	Balanced (best split per node)	Unbalanced (best split per leaf)	Symmetric (same split per level)

Model comparisons

Feature	Random Forest	XGBoost	LightGBM	CatBoost
Core approach	Bagging	Boosting	Boosting	Boosting
Tree growth	Level-wise	Level-wise	Leaf-wise (best-first)	Symmetric (oblivious)
Split behavior	Balanced (best split per node + random features)	Balanced (best split per node)	Unbalanced (best split per leaf)	Symmetric (same split per level)
Categorical features	Manual encoding required	Supported (enable + correct dtype)	Supported (auto-detect via dtype; best to specify)	Built-in (automatic)

Model comparisons

Feature	Random Forest	XGBoost	LightGBM	CatBoost
Core approach	Bagging	Boosting	Boosting	Boosting
Tree growth	Level-wise	Level-wise	Leaf-wise (best-first)	Symmetric (oblivious)
Split behavior	Balanced (best split per node + random features)	Balanced (best split per node)	Unbalanced (best split per leaf)	Symmetric (same split per level)
Categorical features	Manual encoding required	Supported (enable + correct dtype)	Supported (auto-detect via dtype; best to specify)	Built-in (automatic)
Categorical strategy	One-hot	Partitioning (group categories)	Optimized partitioning (grouping)	Ordered target encoding

Model comparisons

Feature	Random Forest	XGBoost	LightGBM	CatBoost
Core approach	Bagging	Boosting	Boosting	Boosting
Tree growth	Level-wise	Level-wise	Leaf-wise (best-first)	Symmetric (oblivious)
Split behavior	Balanced (best split per node + random features)	Balanced (best split per node)	Unbalanced (best split per leaf)	Symmetric (same split per level)
Categorical features	Manual encoding required	Supported (enable + correct dtype)	Supported (auto-detect via dtype; best to specify)	Built-in (automatic)
Categorical strategy	One-hot	Partitioning (group categories)	Optimized partitioning (grouping)	Ordered target encoding
Histogram binning	No	Yes	Yes (more optimized)	Yes

Model comparisons

Feature	Random Forest	XGBoost	LightGBM	CatBoost
Core approach	Bagging	Boosting	Boosting	Boosting
Tree growth	Level-wise	Level-wise	Leaf-wise (best-first)	Symmetric (oblivious)
Split behavior	Balanced (best split per node + random features)	Balanced (best split per node)	Unbalanced (best split per leaf)	Symmetric (same split per level)
Categorical features	Manual encoding required	Supported (enable + correct dtype)	Supported (auto-detect via dtype; best to specify)	Built-in (automatic)
Categorical strategy	One-hot	Partitioning (group categories)	Optimized partitioning (grouping)	Ordered target encoding
Histogram binning	No	Yes	Yes (more optimized)	Yes
Speed (large data)	Moderate	Fast(est)	Fastest (training)	Fast (slower training, fast prediction)

Model comparisons

Feature	Random Forest	XGBoost	LightGBM	CatBoost
Core approach	Bagging	Boosting	Boosting	Boosting
Tree growth	Level-wise	Level-wise	Leaf-wise (best-first)	Symmetric (oblivious)
Split behavior	Balanced (best split per node + random features)	Balanced (best split per node)	Unbalanced (best split per leaf)	Symmetric (same split per level)
Categorical features	Manual encoding required	Supported (enable + correct dtype)	Supported (auto-detect via dtype; best to specify)	Built-in (automatic)
Categorical strategy	One-hot	Partitioning (group categories)	Optimized partitioning (grouping)	Ordered target encoding
Histogram binning	No	Yes	Yes (more optimized)	Yes
Speed (large data)	Moderate	Fast(est)	Fastest (training)	Fast (slower training, fast prediction)
Overfitting risk	Low	Moderate	Higher (needs tuning)	Lower
Early stopping	No	Yes	Yes	Yes

Model comparisons

Feature	Random Forest	XGBoost	LightGBM	CatBoost
Core approach	Bagging	Boosting	Boosting	Boosting
Tree growth	Level-wise	Level-wise	Leaf-wise (best-first)	Symmetric (oblivious)
Split behavior	Balanced (best split per node + random features)	Balanced (best split per node)	Unbalanced (best split per leaf)	Symmetric (same split per level)
Categorical features	Manual encoding required	Supported (enable + correct dtype)	Supported (auto-detect via dtype; best to specify)	Built-in (automatic)
Categorical strategy	One-hot	Partitioning (group categories)	Optimized partitioning (grouping)	Ordered target encoding
Histogram binning	No	Yes	Yes (more optimized)	Yes
Speed (large data)	Moderate	Fast(est)	Fastest (training)	Fast (slower training, fast prediction)
Overfitting risk	Low	Moderate	Higher (needs tuning)	Lower
Early stopping	No	Yes	Yes	Yes
Class imbalance	Manual	Scale_pos_weight	ls_unbalance /class_weight	Class_weights

Model comparisons

Feature	Random Forest	XGBoost	LightGBM	CatBoost
Core approach	Bagging	Boosting	Boosting	Boosting
Tree growth	Level-wise	Level-wise	Leaf-wise (best-first)	Symmetric (oblivious)
Split behavior	Balanced (best split per node + random features)	Balanced (best split per node)	Unbalanced (best split per leaf)	Symmetric (same split per level)
Categorical features	Manual encoding required	Supported (enable + correct dtype)	Supported (auto-detect via dtype; best to specify)	Built-in (automatic)
Categorical strategy	One-hot	Partitioning (group categories)	Optimized partitioning (grouping)	Ordered target encoding
Histogram binning	No	Yes	Yes (more optimized)	Yes
Speed (large data)	Moderate	Fast(est)	Fastest (training)	Fast (slower training, fast prediction)
Overfitting risk	Low	Moderate	Higher (needs tuning)	Lower
Early stopping	No	Yes	Yes	Yes (numeric)
Class imbalance	Class_weight = balanced	Scale_pos_weight (manual calc)	Is_unbalance (auto) class_weight (manual)	Class_weights = [] (manual)
Best use case	Small/noisy data; strong baseline	Reliable general tabular model	Very large datasets; training speed critical	Categorical-heavy data; fast predictions (production)

Tuning with Optuna

```
# build xgboost using scale_pos_weight to handle class imbalance and tune using optuna

def objective(trial):

    max_depth = trial.suggest_int('max_depth', 3, 10)
    learning_rate = trial.suggest_float('learning_rate', 0.01, 0.1)
    subsample = trial.suggest_float('subsample', 0.8, 1.0)

    xgb_optuna = XGBClassifier(random_state=42, scale_pos_weight=scale_pos_weight, max_depth=max_depth,
                               learning_rate=learning_rate, subsample=subsample)
    xgb_optuna.fit(X_train, y_train)
    y_pred_optuna = xgb_optuna.predict(X_test)
    return f1_score(y_test, y_pred_optuna)

study = optuna.create_study(direction='maximize')

study.optimize(objective, n_trials=50)

print("Best XGBoost Parameters from Optuna:", study.best_params)

best_xgb_optuna = XGBClassifier(random_state=42, scale_pos_weight=scale_pos_weight, **study.best_params)
best_xgb_optuna.fit(X_train, y_train)
y_pred_best_xgb_optuna = best_xgb_optuna.predict(X_test)
print("Best XGBoost from Optuna F1 Score:", f1_score(y_test, y_pred_best_xgb_optuna))
```

Loading...